

Guía Completa PostgreSQL

De Junior a DBA - Triggers, Funciones, Replicación y Más



Tabla de Contenidos

1. [Funciones en PostgreSQL](#)
 2. [Triggers \(Disparadores\)](#)
 3. [Replicación Lógica \(Pub/Sub\)](#)
 4. [Replicación Física \(Streaming\)](#)
 5. [Procedures \(Procedimientos Almacenados\)](#)
 6. [Índices y Optimización](#)
 7. [Particionamiento](#)
 8. [Backup y Recuperación](#)
 9. [Seguridad y Permisos](#)
 10. [Monitoreo y Mantenimiento](#)
-

Funciones en PostgreSQL

Las funciones son bloques de código reutilizables que aceptan parámetros y retornan valores.

Anatomía de una Función

```
sql
```

```
CREATE OR REPLACE FUNCTION nombre_funcion(  
    parametro1 tipo1,  
    parametro2 tipo2  
) RETURNS tipo_retorno AS $$  
DECLARE  
    -- Variables locales  
    variable1 tipo;  
    variable2 tipo;  
BEGIN  
    -- Lógica de la función  
    -- ...  
    RETURN valor;  
END;  
$$ LANGUAGE plpgsql;
```

Tipos de Funciones

1. Función Escalar (retorna un valor único)

```
sql  
  
-- Función simple: calcular IVA  
CREATE OR REPLACE FUNCTION calcular_iva(monto DECIMAL)  
RETURNS DECIMAL AS $$  
BEGIN  
    RETURN monto * 0.15; -- 15% de IVA  
END;  
$$ LANGUAGE plpgsql;  
  
-- Uso  
SELECT calcular_iva(100); -- Retorna: 15.00
```

2. Función con Múltiples Parámetros

```
sql
```

```

-- Calcular total con descuento e IVA
CREATE OR REPLACE FUNCTION calcular_total(
    subtotal DECIMAL,
    descuento_porcentaje DECIMAL DEFAULT 0,
    aplicar_iva BOOLEAN DEFAULT true
) RETURNS DECIMAL AS $$
DECLARE
    monto_descuento DECIMAL;
    monto_con_descuento DECIMAL;
    total DECIMAL;
BEGIN
    -- Aplicar descuento
    monto_descuento := subtotal * (descuento_porcentaje / 100);
    monto_con_descuento := subtotal - monto_descuento;

    -- Aplicar IVA si corresponde
    IF aplicar_iva THEN
        total := monto_con_descuento * 1.15;
    ELSE
        total := monto_con_descuento;
    END IF;

    RETURN ROUND(total, 2);
END;
$$ LANGUAGE plpgsql;

-- Usos
SELECT calcular_total(100);           -- 115.00 (con IVA)
SELECT calcular_total(100, 10);       -- 103.50 (10% desc + IVA)
SELECT calcular_total(100, 10, false); -- 90.00 (10% desc, sin IVA)

```

3. Función que Retorna Tabla (TABLE)

```
sql
```

-- Obtener ventas del mes actual

```
CREATE OR REPLACE FUNCTION ventas_mes_actual()
RETURNS TABLE(
    venta_id INTEGER,
    cliente_nombre VARCHAR,
    total DECIMAL,
    fecha DATE
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        v.id,
        c.nombre,
        v.total,
        v.fecha::DATE
    FROM ventas v
    JOIN clientes c ON v.cliente_id = c.id
    WHERE EXTRACT(MONTH FROM v.fecha) = EXTRACT(MONTH FROM CURRENT_DATE)
        AND EXTRACT(YEAR FROM v.fecha) = EXTRACT(YEAR FROM CURRENT_DATE);
END;
$$ LANGUAGE plpgsql;

-- Uso
SELECT * FROM ventas_mes_actual();
```

4. Función que Retorna SETOF (conjunto de filas)

sql

-- Obtener productos por categoría

```
CREATE OR REPLACE FUNCTION productos_por_categoria(cat_id INTEGER)
RETURNS SETOF productos AS $$
BEGIN
    RETURN QUERY
    SELECT * FROM productos
    WHERE categoria_id = cat_id
    ORDER BY nombre;
END;
$$ LANGUAGE plpgsql;

-- Uso
SELECT * FROM productos_por_categoria(5);
```

5. Función con Tipo de Retorno Compuesto

sql

-- Definir tipo personalizado

```
CREATE TYPE resumen_cliente AS (  
    total_compras INTEGER,  
    monto_total DECIMAL,  
    promedio_compra DECIMAL,  
    ultima_compra DATE  
);
```

-- Función que retorna el tipo compuesto

```
CREATE OR REPLACE FUNCTION resumen_cliente(cliente_id INTEGER)  
RETURNS resumen_cliente AS $$  
DECLARE  
    resultado resumen_cliente;  
BEGIN  
    SELECT  
        COUNT(*),  
        COALESCE(SUM(total), 0),  
        COALESCE(AVG(total), 0),  
        MAX(fecha)::DATE  
    INTO resultado  
    FROM ventas  
    WHERE ventas.cliente_id = resumen_cliente.cliente_id;  
  
    RETURN resultado;  
END;  
$$ LANGUAGE plpgsql;
```

-- Uso

```
SELECT * FROM resumen_cliente(10);
```

-- Acceder a campos específicos

```
SELECT (resumen_cliente(10)).total_compras;  
SELECT (resumen_cliente(10)).monto_total;
```

Estructuras de Control

IF-ELSIF-ELSE

sql

```
CREATE OR REPLACE FUNCTION clasificar_venta(monto DECIMAL)
RETURNS VARCHAR AS $$
BEGIN
    IF monto < 100 THEN
        RETURN 'PEQUEÑA';
    ELSIF monto < 1000 THEN
        RETURN 'MEDIANA';
    ELSIF monto < 10000 THEN
        RETURN 'GRANDE';
    ELSE
        RETURN 'CORPORATIVA';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

CASE

```
sql

CREATE OR REPLACE FUNCTION dia_semana_es(fecha DATE)
RETURNS VARCHAR AS $$
BEGIN
    RETURN CASE EXTRACT(DOW FROM fecha)
        WHEN 0 THEN 'Domingo'
        WHEN 1 THEN 'Lunes'
        WHEN 2 THEN 'Martes'
        WHEN 3 THEN 'Miércoles'
        WHEN 4 THEN 'Jueves'
        WHEN 5 THEN 'Viernes'
        WHEN 6 THEN 'Sábado'
    END;
END;
$$ LANGUAGE plpgsql;
```

LOOP (Bucles)

```
sql
```

-- LOOP simple con EXIT

```
CREATE OR REPLACE FUNCTION generar_numeros(limite INTEGER)
RETURNS TABLE(numero INTEGER) AS $$
DECLARE
    i INTEGER := 1;
BEGIN
    LOOP
        numero := i;
        RETURN NEXT;
        i := i + 1;
        EXIT WHEN i > limite;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

-- FOR loop

```
CREATE OR REPLACE FUNCTION suma_hasta(n INTEGER)
RETURNS INTEGER AS $$
DECLARE
    suma INTEGER := 0;
    i INTEGER;
BEGIN
    FOR i IN 1..n LOOP
        suma := suma + i;
    END LOOP;
    RETURN suma;
END;
$$ LANGUAGE plpgsql;
```

-- WHILE loop

```
CREATE OR REPLACE FUNCTION factorial(n INTEGER)
RETURNS BIGINT AS $$
DECLARE
    resultado BIGINT := 1;
    contador INTEGER := n;
BEGIN
    WHILE contador > 1 LOOP
        resultado := resultado * contador;
        contador := contador - 1;
    END LOOP;
    RETURN resultado;
END;
$$ LANGUAGE plpgsql;
```

-- FOR loop sobre query result

```
CREATE OR REPLACE FUNCTION listar_clientes_activos()
```

```
RETURNS VOID AS $$
```

```
DECLARE
```

```
    cliente RECORD;
```

```
BEGIN
```

```
    FOR cliente IN
```

```
        SELECT id, nombre, email FROM clientes WHERE activo = true
```

```
    LOOP
```

```
        RAISE NOTICE 'Cliente %: % (%)', cliente.id, cliente.nombre, cliente.email;
```

```
    END LOOP;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

Manejo de Excepciones

```
sql
```



```

CREATE OR REPLACE FUNCTION dividir_seguro(
    numerador DECIMAL,
    denominador DECIMAL
) RETURNS DECIMAL AS $$
DECLARE
    resultado DECIMAL;
BEGIN
    resultado := numerador / denominador;
    RETURN resultado;
EXCEPTION
    WHEN division_by_zero THEN
        RAISE NOTICE 'Error: División por cero';
        RETURN NULL;
    WHEN OTHERS THEN
        RAISE NOTICE 'Error inesperado: %', SQLERRM;
        RETURN NULL;
END;
$$ LANGUAGE plpgsql;

-- Función con manejo completo de errores
CREATE OR REPLACE FUNCTION crear_cliente_seguro(
    p_nombre VARCHAR,
    p_email VARCHAR
) RETURNS INTEGER AS $$
DECLARE
    nuevo_id INTEGER;
BEGIN
    INSERT INTO clientes (nombre, email, fecha_registro)
    VALUES (p_nombre, p_email, NOW())
    RETURNING id INTO nuevo_id;

    RETURN nuevo_id;
EXCEPTION
    WHEN unique_violation THEN
        RAISE EXCEPTION 'El email % ya está registrado', p_email;
    WHEN not_null_violation THEN
        RAISE EXCEPTION 'Nombre y email son obligatorios';
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Error al crear cliente: %', SQLERRM;
END;
$$ LANGUAGE plpgsql;

```

Funciones con OUT Parameters

sql

-- Retornar múltiples valores sin definir tipo compuesto

```
CREATE OR REPLACE FUNCTION estadisticas_ventas(  
    fecha_inicio DATE,  
    fecha_fin DATE,  
    OUT total_ventas INTEGER,  
    OUT monto_total DECIMAL,  
    OUT promedio DECIMAL  
) AS $$  
BEGIN  
    SELECT  
        COUNT(*),  
        COALESCE(SUM(total), 0),  
        COALESCE(AVG(total), 0)  
    INTO total_ventas, monto_total, promedio  
    FROM ventas  
    WHERE fecha BETWEEN fecha_inicio AND fecha_fin;  
END;  
$$ LANGUAGE plpgsql;
```

-- Uso

```
SELECT * FROM estadisticas_ventas('2025-01-01', '2025-12-31');
```

-- O acceder individualmente

```
SELECT total_ventas, monto_total  
FROM estadisticas_ventas('2025-01-01', '2025-12-31');
```

Triggers (Disparadores)

Los triggers ejecutan funciones automáticamente en respuesta a eventos (INSERT, UPDATE, DELETE).

Anatomía de un Trigger

```
sql
```

```

-- 1. Crear la función trigger
CREATE OR REPLACE FUNCTION nombre_funcion_trigger()
RETURNS TRIGGER AS $$
BEGIN
    -- Lógica del trigger
    -- NEW: nueva fila (INSERT, UPDATE)
    -- OLD: fila antigua (UPDATE, DELETE)
    RETURN NEW; -- o OLD, o NULL según el caso
END;
$$ LANGUAGE plpgsql;

-- 2. Crear el trigger
CREATE TRIGGER nombre_trigger
    BEFORE/AFTER INSERT/UPDATE/DELETE
    ON nombre_tabla
    FOR EACH ROW
    EXECUTE FUNCTION nombre_funcion_trigger();

```

Variables Especiales en Triggers

- **NEW:** Fila nueva (disponible en INSERT y UPDATE)
- **OLD:** Fila antigua (disponible en UPDATE y DELETE)
- **TG_OP:** Operación que disparó el trigger ('INSERT', 'UPDATE', 'DELETE')
- **TG_TABLE_NAME:** Nombre de la tabla
- **TG_WHEN:** 'BEFORE' o 'AFTER'
- **TG_LEVEL:** 'ROW' o 'STATEMENT'

Ejemplos Prácticos

1. Trigger BEFORE INSERT - Validación y Modificación

```
sql
```

-- Tabla de ejemplo

```
CREATE TABLE productos (  
  id SERIAL PRIMARY KEY,  
  codigo VARCHAR(50) UNIQUE NOT NULL,  
  nombre VARCHAR(200) NOT NULL,  
  precio DECIMAL(10,2) NOT NULL,  
  stock INTEGER DEFAULT 0,  
  fecha_creacion TIMESTAMP,  
  fecha_modificacion TIMESTAMP  
);
```

-- Trigger: Auto-generar código, validar precio, timestamps

```
CREATE OR REPLACE FUNCTION trg_producto_before_insert()  
RETURNS TRIGGER AS $$  
BEGIN  
  -- Auto-generar código si está vacío  
  IF NEW.codigo IS NULL OR NEW.codigo = '' THEN  
    NEW.codigo := 'PROD-' || LPAD(nextval('productos_id_seq')::TEXT, 6, '0');  
  END IF;  
  
  -- Validar que el precio sea positivo  
  IF NEW.precio <= 0 THEN  
    RAISE EXCEPTION 'El precio debe ser mayor a cero';  
  END IF;  
  
  -- Normalizar nombre (quitar espacios extras, capitalizar)  
  NEW.nombre := INITCAP(TRIM(NEW.nombre));  
  
  -- Establecer fecha de creación  
  NEW.fecha_creacion := NOW();  
  NEW.fecha_modificacion := NOW();  
  
  RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER producto_before_insert  
BEFORE INSERT ON productos  
FOR EACH ROW  
EXECUTE FUNCTION trg_producto_before_insert();
```

-- Prueba

```
INSERT INTO productos (nombre, precio, stock)  
VALUES ('laptop dell', 850.50, 10);
```

-- Resultado: código auto-generado, nombre capitalizado, fechas automáticas

2. Trigger BEFORE UPDATE - Auditoría de Cambios

sql

-- Tabla de auditoría

```
CREATE TABLE productos_auditoria (  
  id SERIAL PRIMARY KEY,  
  producto_id INTEGER,  
  campo_modificado VARCHAR(100),  
  valor_anterior TEXT,  
  valor_nuevo TEXT,  
  usuario VARCHAR(100),  
  fecha_modificacion TIMESTAMP DEFAULT NOW()  
);
```

-- Trigger que registra cambios

```
CREATE OR REPLACE FUNCTION trg_producto_before_update()  
RETURNS TRIGGER AS $$  
BEGIN
```

-- Actualizar fecha de modificación

```
NEW.fecha_modificacion := NOW();
```

-- Registrar cambio de precio

```
IF OLD.precio IS DISTINCT FROM NEW.precio THEN  
  INSERT INTO productos_auditoria (  
    producto_id, campo_modificado, valor_anterior, valor_nuevo, usuario  
  ) VALUES (  
    OLD.id, 'precio', OLD.precio::TEXT, NEW.precio::TEXT, current_user  
  );  
END IF;
```

-- Registrar cambio de stock

```
IF OLD.stock IS DISTINCT FROM NEW.stock THEN  
  INSERT INTO productos_auditoria (  
    producto_id, campo_modificado, valor_anterior, valor_nuevo, usuario  
  ) VALUES (  
    OLD.id, 'stock', OLD.stock::TEXT, NEW.stock::TEXT, current_user  
  );  
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER producto_before_update  
BEFORE UPDATE ON productos  
FOR EACH ROW  
EXECUTE FUNCTION trg_producto_before_update();
```

-- Prueba

```
UPDATE productos SET precio = 899.99, stock = 15 WHERE id = 1;
```

```
-- Ver auditoría
```

```
SELECT * FROM productos_auditoria WHERE producto_id = 1;
```

3. Trigger AFTER INSERT - Acciones en Cascada

```
sql
```

-- Tablas de ejemplo

```
CREATE TABLE ventas (  
  id SERIAL PRIMARY KEY,  
  cliente_id INTEGER NOT NULL,  
  total DECIMAL(10,2) NOT NULL,  
  fecha TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE clientes (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100),  
  total_compras INTEGER DEFAULT 0,  
  monto_total_comprado DECIMAL(10,2) DEFAULT 0,  
  ultima_compra TIMESTAMP  
);
```

-- Trigger: Actualizar estadísticas del cliente

```
CREATE OR REPLACE FUNCTION trg_venta_after_insert()  
RETURNS TRIGGER AS $$  
BEGIN  
  UPDATE clientes  
  SET  
    total_compras = total_compras + 1,  
    monto_total_comprado = monto_total_comprado + NEW.total,  
    ultima_compra = NEW.fecha  
  WHERE id = NEW.cliente_id;  
  
  RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER venta_after_insert  
AFTER INSERT ON ventas  
FOR EACH ROW  
EXECUTE FUNCTION trg_venta_after_insert();
```

-- Prueba

```
INSERT INTO ventas (cliente_id, total) VALUES (5, 150.75);
```

-- El cliente 5 automáticamente actualiza sus estadísticas

4. Trigger BEFORE DELETE - Soft Delete

sql

-- Agregar columna de borrado lógico

```
ALTER TABLE productos ADD COLUMN eliminado BOOLEAN DEFAULT false;
```

```
ALTER TABLE productos ADD COLUMN fecha_eliminacion TIMESTAMP;
```

-- Trigger: Convertir DELETE en UPDATE (soft delete)

```
CREATE OR REPLACE FUNCTION trg_producto_soft_delete()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

-- En lugar de eliminar, marcar como eliminado

```
UPDATE productos
```

```
SET
```

```
    eliminado = true,
```

```
    fecha_eliminacion = NOW()
```

```
WHERE id = OLD.id;
```

-- Retornar NULL para cancelar el DELETE real

```
RETURN NULL;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER producto_soft_delete
```

```
BEFORE DELETE ON productos
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION trg_producto_soft_delete();
```

-- Prueba

```
DELETE FROM productos WHERE id = 5;
```

-- En realidad no se borra, solo se marca eliminado = true

-- Ver productos activos

```
SELECT * FROM productos WHERE eliminado = false;
```

5. Trigger Multi-Operación (INSERT, UPDATE, DELETE)

sql

-- Tabla de log

```
CREATE TABLE cambios_log (  
  id SERIAL PRIMARY KEY,  
  tabla VARCHAR(100),  
  operacion VARCHAR(20),  
  registro_id INTEGER,  
  datos_anteriores JSONB,  
  datos_nuevos JSONB,  
  usuario VARCHAR(100),  
  fecha TIMESTAMP DEFAULT NOW()  
);
```

-- Trigger universal

```
CREATE OR REPLACE FUNCTION trg_registrar_cambios()  
RETURNS TRIGGER AS $$  
DECLARE  
  datos_old JSONB;  
  datos_new JSONB;  
BEGIN  
  -- Convertir filas a JSON  
  IF TG_OP = 'DELETE' THEN  
    datos_old := row_to_json(OLD)::JSONB;  
    datos_new := NULL;  
  ELSIF TG_OP = 'UPDATE' THEN  
    datos_old := row_to_json(OLD)::JSONB;  
    datos_new := row_to_json(NEW)::JSONB;  
  ELSIF TG_OP = 'INSERT' THEN  
    datos_old := NULL;  
    datos_new := row_to_json(NEW)::JSONB;  
  END IF;
```

-- Registrar el cambio

```
INSERT INTO cambios_log (  
  tabla, operacion, registro_id, datos_anteriores, datos_nuevos, usuario  
) VALUES (  
  TG_TABLE_NAME,  
  TG_OP,  
  COALESCE(NEW.id, OLD.id),  
  datos_old,  
  datos_new,  
  current_user  
);
```

-- Retornar según operación

```
IF TG_OP = 'DELETE' THEN  
  RETURN OLD;
```

```
ELSE
    RETURN NEW;
END IF;
END;
$$ LANGUAGE plpgsql;

-- Aplicar a múltiples tablas
CREATE TRIGGER clientes_log
    AFTER INSERT OR UPDATE OR DELETE ON clientes
    FOR EACH ROW
    EXECUTE FUNCTION trg_registrar_cambios();

CREATE TRIGGER productos_log
    AFTER INSERT OR UPDATE OR DELETE ON productos
    FOR EACH ROW
    EXECUTE FUNCTION trg_registrar_cambios();
```

6. Trigger para Mantener Integridad (Stock)

```
sql
```

```
CREATE TABLE ventas_detalle (  
  id SERIAL PRIMARY KEY,  
  venta_id INTEGER,  
  producto_id INTEGER,  
  cantidad INTEGER,  
  precio_unitario DECIMAL(10,2)  
);
```

-- Trigger: Descontar stock al vender

```
CREATE OR REPLACE FUNCTION trg_venta_detalle_descontar_stock()  
RETURNS TRIGGER AS $$  
DECLARE  
  stock_actual INTEGER;  
BEGIN  
  -- Obtener stock actual  
  SELECT stock INTO stock_actual  
  FROM productos  
  WHERE id = NEW.producto_id;  
  
  -- Validar que hay stock suficiente  
  IF stock_actual < NEW.cantidad THEN  
    RAISE EXCEPTION 'Stock insuficiente. Disponible: %, Solicitado: %',  
      stock_actual, NEW.cantidad;  
  END IF;  
  
  -- Descontar stock  
  UPDATE productos  
  SET stock = stock - NEW.cantidad  
  WHERE id = NEW.producto_id;  
  
  RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER venta_detalle_after_insert  
AFTER INSERT ON ventas_detalle  
FOR EACH ROW  
EXECUTE FUNCTION trg_venta_detalle_descontar_stock();
```

-- Trigger: Devolver stock al cancelar venta

```
CREATE OR REPLACE FUNCTION trg_venta_detalle_devolver_stock()  
RETURNS TRIGGER AS $$  
BEGIN  
  UPDATE productos  
  SET stock = stock + OLD.cantidad  
  WHERE id = OLD.producto_id;
```

```

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER venta_detalle_after_delete
AFTER DELETE ON ventas_detalle
FOR EACH ROW
EXECUTE FUNCTION trg_venta_detalle_devolver_stock();

```

Statement-Level Triggers

```

sql

-- Trigger que se ejecuta UNA VEZ por statement (no por cada fila)
CREATE OR REPLACE FUNCTION trg_productos_statement()
RETURNS TRIGGER AS $$
BEGIN
    RAISE NOTICE 'Se ejecutó % en la tabla productos', TG_OP;

    IF TG_OP = 'INSERT' THEN
        RAISE NOTICE 'Se insertaron filas en productos';
    ELSIF TG_OP = 'UPDATE' THEN
        RAISE NOTICE 'Se actualizaron filas en productos';
    END IF;

    RETURN NULL; -- En AFTER statement, el retorno se ignora
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER productos_statement_trigger
AFTER INSERT OR UPDATE ON productos
FOR EACH STATEMENT
EXECUTE FUNCTION trg_productos_statement();

-- Al ejecutar: INSERT INTO productos ... (10 filas)
-- El trigger se ejecuta UNA sola vez, no 10 veces

```

Deshabilitar/Habilitar Triggers

```

sql

```

```
-- Deshabilitar un trigger específico
ALTER TABLE productos DISABLE TRIGGER producto_before_insert;

-- Habilitar
ALTER TABLE productos ENABLE TRIGGER producto_before_insert;

-- Deshabilitar TODOS los triggers de una tabla
ALTER TABLE productos DISABLE TRIGGER ALL;

-- Habilitar todos
ALTER TABLE productos ENABLE TRIGGER ALL;

-- Ver triggers de una tabla
SELECT
    trigger_name,
    event_manipulation,
    event_object_table,
    action_timing
FROM information_schema.triggers
WHERE event_object_table = 'productos';
```

Replicación Lógica (Pub/Sub)

La replicación lógica permite replicar datos a nivel de tabla usando publicaciones y suscripciones.

Conceptos Clave

- **Publisher (Publicador):** Base de datos que ENVÍA cambios
- **Subscriber (Suscriptor):** Base de datos que RECIBE cambios
- **Publication:** Define QUÉ tablas se replican
- **Subscription:** Se suscribe a una publication
- **Replication Slot:** Mantiene el tracking de cambios

Ventajas vs Replicación Física

Característica	Lógica (Pub/Sub)	Física (Streaming)
Nivel	Tabla/Datos	Disco/WAL completo
Selectividad	Puedes elegir tablas	Todo o nada
Versiones PostgreSQL	Pueden ser diferentes	Deben ser iguales

Característica	Lógica (Pub/Sub)	Física (Streaming)
DDL (ALTER TABLE)	No se replica	Sí se replica
Dirección	Puede ser bi-direccional	Solo master→replica
Overhead	Mayor	Menor

Configuración de Replicación Lógica

Pre-requisitos

```
sql

-- En AMBAS bases de datos (Publisher y Subscriber)

-- 1. Verificar/Configurar postgresql.conf
wal_level = logical           # Debe ser 'logical'
max_replication_slots = 10    # Número de slots
max_wal_senders = 10          # Número de procesos WAL sender

-- 2. Configurar pg_hba.conf (en Publisher)
-- Permitir conexiones de replicación desde Subscriber
# TYPE DATABASE  USER        ADDRESS          METHOD
host replication replicador  192.168.1.0/24   md5
host all         replicador  192.168.1.0/24   md5

-- 3. Reiniciar PostgreSQL
sudo systemctl restart postgresql
```

Setup Completo - Paso a Paso

PASO 1: Preparar el Publisher (Base de Datos Origen)

```
sql
```

-- Conectar a la BD Publisher (ejemplo: quito_db)

\c quito_db

-- Crear usuario de replicación

CREATE USER replicador WITH REPLICATION PASSWORD 'replica123';

-- Dar permisos sobre las tablas a replicar

GRANT SELECT ON ALL TABLES IN SCHEMA public TO replicador;

GRANT USAGE ON SCHEMA public TO replicador;

-- Crear las tablas que se replicarán

```
CREATE TABLE clientes (  
    id SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    fecha_registro TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE productos (  
    id SERIAL PRIMARY KEY,  
    codigo VARCHAR(50) UNIQUE NOT NULL,  
    nombre VARCHAR(200) NOT NULL,  
    precio DECIMAL(10,2) NOT NULL,  
    stock INTEGER DEFAULT 0  
);
```

-- Insertar datos de prueba

```
INSERT INTO clientes (nombre, email) VALUES  
(  
'Juan Pérez', 'juan@email.com'),  
(  
'María López', 'maria@email.com'),  
(  
'Carlos Ruiz', 'carlos@email.com');
```

```
INSERT INTO productos (codigo, nombre, precio, stock) VALUES  
(  
'PROD-001', 'Laptop Dell', 850.00, 10),  
(  
'PROD-002', 'Mouse Logitech', 25.50, 50),  
(  
'PROD-003', 'Teclado Mecánico', 120.00, 30);
```

-- CREAR LA PUBLICATION (QUÉ se replica)

CREATE PUBLICATION pub_principal FOR ALL TABLES;

-- O publicar solo tablas específicas:

-- CREATE PUBLICATION pub_principal FOR TABLE clientes, productos;

-- Ver publications

SELECT * FROM pg_publication;


```
-- Ver qué tablas están en la publication
```

```
SELECT * FROM pg_publication_tables WHERE pubname = 'pub_principal';
```

PASO 2: Preparar el Subscriber (Base de Datos Destino)

```
sql
```

```
-- Conectar a la BD Subscriber (ejemplo: guayaquil_db)
```

```
\c guayaquil_db
```

```
-- IMPORTANTE: Crear la MISMA estructura de tablas (sin datos)
```

```
CREATE TABLE clientes (  
    id SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    fecha_registro TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE productos (  
    id SERIAL PRIMARY KEY,  
    codigo VARCHAR(50) UNIQUE NOT NULL,  
    nombre VARCHAR(200) NOT NULL,  
    precio DECIMAL(10,2) NOT NULL,  
    stock INTEGER DEFAULT 0  
);
```

```
-- NOTA: Las tablas DEBEN tener la misma estructura, pero pueden:
```

```
-- - Tener indices diferentes
```

```
-- - Tener constraints diferentes (aunque se recomienda mantenerlos iguales)
```

```
-- CREAR LA SUBSCRIPTION
```

```
CREATE SUBSCRIPTION sub_guayaquil  
CONNECTION 'host=postgres-quito port=5432 dbname=quito_db user=replicador password=replica123'  
PUBLICATION pub_principal  
WITH (copy_data = true); -- copy_data = true hace carga inicial de datos existentes
```

```
-- Ver subscriptions
```

```
SELECT * FROM pg_subscription;
```

```
-- Ver estado de replicación
```

```
SELECT * FROM pg_stat_subscription;
```

PASO 3: Verificar que Funciona

```
sql
```

```
-- En PUBLISHER (quito_db), insertar datos
INSERT INTO clientes (nombre, email)
VALUES ('Nuevo Cliente', 'nuevo@email.com');

-- En SUBSCRIBER (guayaquil_db), verificar (esperar 1-2 segundos)
SELECT * FROM clientes WHERE email = 'nuevo@email.com';
-- Debería aparecer el nuevo cliente

-- Probar UPDATE
-- En PUBLISHER
UPDATE productos SET precio = 899.99 WHERE codigo = 'PROD-001';

-- En SUBSCRIBER
SELECT precio FROM productos WHERE codigo = 'PROD-001';
-- Debería mostrar 899.99

-- Probar DELETE
-- En PUBLISHER
DELETE FROM clientes WHERE email = 'carlos@email.com';

-- En SUBSCRIBER
SELECT * FROM clientes WHERE email = 'carlos@email.com';
-- No debería existir
```

Escenarios Avanzados

1. Replicación Selectiva (Solo Algunas Tablas)

```
sql

-- En PUBLISHER
CREATE PUBLICATION pub_solo_clientes FOR TABLE clientes;

-- En SUBSCRIBER
CREATE SUBSCRIPTION sub_solo_clientes
CONNECTION 'host=postgres-quito port=5432 dbname=quito_db user=replicador password=replica123'
PUBLICATION pub_solo_clientes;
```

2. Replicación con Filtro (Row Filter)

```
sql
```

```
-- PostgreSQL 15+ soporta filtros WHERE en publications
```

```
-- En PUBLISHER: Solo replicar clientes activos
```

```
CREATE PUBLICATION pub_clientes_activos
```

```
FOR TABLE clientes
```

```
WHERE (activo = true);
```

```
-- En SUBSCRIBER
```

```
CREATE SUBSCRIPTION sub_clientes_activos
```

```
CONNECTION '...'
```

```
PUBLICATION pub_clientes_activos;
```

3. Múltiples Publications en una Subscription

```
sql
```

```
-- En PUBLISHER
```

```
CREATE PUBLICATION pub_clientes FOR TABLE clientes;
```

```
CREATE PUBLICATION pub_productos FOR TABLE productos;
```

```
-- En SUBSCRIBER: suscribirse a ambas
```

```
CREATE SUBSCRIPTION sub_completa
```

```
CONNECTION '...'
```

```
PUBLICATION pub_clientes, pub_productos;
```

4. Replicación Bidireccional (Multi-Master)

⚠ **CUIDADO:** Requiere manejo cuidadoso de conflictos

```
sql
```

-- Base de Datos A (Quito)

-- 1. Crear publication

CREATE PUBLICATION pub_quito FOR ALL TABLES;

-- 2. Crear subscription a Base B

CREATE SUBSCRIPTION sub_desde_guayaquil

CONNECTION 'host=postgres-guayaquil port=5432 dbname=guayaquil_db user=replicador password=replica123'

PUBLICATION pub_guayaquil;

-- Base de Datos B (Guayaquil)

-- 1. Crear publication

CREATE PUBLICATION pub_guayaquil FOR ALL TABLES;

-- 2. Crear subscription a Base A

CREATE SUBSCRIPTION sub_desde_quito

CONNECTION 'host=postgres-quito port=5432 dbname=quito_db user=replicador password=replica123'

PUBLICATION pub_quito;

-- IMPORTANTE: Ambas subscriptions deben tener copy_data = false

-- excepto en el setup inicial

-- Para evitar loops infinitos, configurar:

ALTER SUBSCRIPTION sub_desde_guayaquil SET (origin = none);

ALTER SUBSCRIPTION sub_desde_quito SET (origin = none);

Mantenimiento y Monitoreo

sql

-- Ver estado de replicación

SELECT

```
subname AS subscription_name,  
pid,  
received_lsn,  
latest_end_lsn,  
latest_end_time,  
last_msg_send_time,  
last_msg_receipt_time
```

FROM pg_stat_subscription;

-- Ver replication slots

SELECT * FROM pg_replication_slots;

-- Ver lag de replicación

SELECT

```
slot_name,  
confirmed_flush_lsn,  
pg_current_wal_lsn(),  
(pg_current_wal_lsn() - confirmed_flush_lsn) AS lag_bytes
```

FROM pg_replication_slots

WHERE slot_type = 'logical';

-- Pausar replicación temporalmente

ALTER SUBSCRIPTION sub_guayaquil DISABLE;

-- Reanudar

ALTER SUBSCRIPTION sub_guayaquil ENABLE;

-- Refrescar (resincronizar) una subscription

ALTER SUBSCRIPTION sub_guayaquil REFRESH PUBLICATION;

-- Eliminar subscription

DROP SUBSCRIPTION sub_guayaquil;

-- Eliminar publication

DROP PUBLICATION pub_principal;

-- Ver conflictos (si los hay)

SELECT * FROM pg_stat_subscription_stats;

Troubleshooting Pub/Sub

Problema: "ERROR: could not create replication slot"

Causa: No hay slots disponibles

Solución:

```
sql

-- Aumentar max_replication_slots en postgresql.conf
max_replication_slots = 20

-- Reiniciar PostgreSQL
```

Problema: "Permission denied for table"

Causa: El usuario replicador no tiene permisos

Solución:

```
sql

-- En PUBLISHER
GRANT SELECT ON ALL TABLES IN SCHEMA public TO replicador;
GRANT USAGE ON SCHEMA public TO replicador;
```

Problema: Los datos no se replican

Checklist:

1. Verificar que wal_level = logical
2. Verificar pg_hba.conf permite conexión
3. Verificar estado: `(SELECT * FROM pg_stat_subscription;)`
4. Revisar logs de PostgreSQL

```
sql

-- Ver si la subscription está activa
SELECT
    subname,
    subconninfo,
    subenabled
FROM pg_subscription;

-- Ver workers activos
SELECT * FROM pg_stat_activity
WHERE application_name LIKE '%subscription%';
```

Replicación Física (Streaming Replication)

Replica TODA la instancia PostgreSQL a nivel de disco (WAL logs).

Streaming Replication vs Logical Replication

Usa Streaming cuando:

- Necesitas réplica exacta (disaster recovery)
- Quieres alta disponibilidad (failover automático)
- Performance es crítica (menor overhead)
- No necesitas filtrar tablas

Usa Logical cuando:

- Solo necesitas algunas tablas
- Versiones diferentes de PostgreSQL
- Necesitas bi-direccional
- Quieres más control sobre QUÉ se replica

Setup de Streaming Replication

PASO 1: Configurar el Master (Primary)

```
bash
```

```
# Editar postgresql.conf
```

```
wal_level = replica          # Nivel WAL para replicación
```

```
max_wal_senders = 3          # Número de réplicas simultáneas
```

```
wal_keep_size = 1GB          # Cuánto WAL mantener (PostgreSQL 13+)
```

```
# wal_keep_segments = 64      # Para PostgreSQL < 13
```

```
# Editar pg_hba.conf
```

```
# Permitir conexiones de replicación desde standby
```

```
# TYPE DATABASE USER ADDRESS METHOD
```

```
host replication replicador 192.168.1.101/32 md5
```

```
host replication replicador 192.168.1.102/32 md5
```

```
sql
```

```
-- Crear usuario de replicación
```

```
CREATE USER replicador WITH REPLICATION PASSWORD 'replica123';
```

```
-- Reiniciar PostgreSQL
```

```
sudo systemctl restart postgresql
```

PASO 2: Crear el Standby (Replica)

```
bash
```

```
# 1. Detener PostgreSQL en el standby (si está corriendo)
```

```
sudo systemctl stop postgresql
```

```
# 2. Limpiar directorio de datos
```

```
sudo rm -rf /var/lib/postgresql/14/main/*
```

```
# 3. Hacer base backup desde el master
```

```
sudo -u postgres pg_basebackup \  
-h 192.168.1.100 \  
-D /var/lib/postgresql/14/main \  
-U replicador \  
-P \  
-v \  
-R \  
-X stream \  
-C -S replica_slot
```

```
# Explicación de flags:
```

```
# -h: Host del master
```

```
# -D: Directorio de datos del standby
```

```
# -U: Usuario de replicación
```

```
# -P: Mostrar progreso
```

```
# -v: Verbose
```

```
# -R: Crear standby.signal y configuración automática
```

```
# -X stream: Streaming de WAL durante backup
```

```
# -C: Crear replication slot
```

```
# -S: Nombre del slot
```

```
# 4. Iniciar PostgreSQL en el standby
```

```
sudo systemctl start postgresql
```

PASO 3: Verificar Estado

```
sql
```



```
-- En el MASTER
SELECT * FROM pg_stat_replication;
-- Deberías ver la conexión del standby

-- Ver replication slots
SELECT * FROM pg_replication_slots;

-- En el STANDBY
SELECT pg_is_in_recovery();
-- Retorna 't' (true) si es standby

-- Ver lag de replicación
SELECT
    now() - pg_last_xact_replay_timestamp() AS replication_lag;
```

Configuración de Standby.signal

```
bash

# En el standby, el archivo standby.signal indica que es una réplica
# Se crea automáticamente con pg_basebackup -R

# Ver configuración en postgresql.auto.conf
cat /var/lib/postgresql/14/main/postgresql.auto.conf

# Debería contener:
primary_conninfo = 'host=192.168.1.100 port=5432 user=replicador password=replica123'
primary_slot_name = 'replica_slot'
```

Promover Standby a Master (Failover)

```
bash

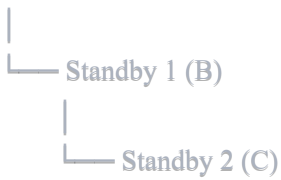
# Si el master falla, promover standby a master
sudo -u postgres pg_ctl promote -D /var/lib/postgresql/14/main

# O desde PostgreSQL
SELECT pg_promote();

# Verificar que ya no es standby
SELECT pg_is_in_recovery(); -- Debería retornar 'f' (false)
```

Replicación Cascading (Cadena)

Master (A)



```
bash

# Standby 1 (B) se conecta a Master (A) normalmente

# Standby 2 (C) se conecta a Standby 1 (B)
# En postgresql.auto.conf de C:
primary_conninfo = 'host=standby1 port=5432 user=replicador password=replica123'
```

Procedures (Procedimientos Almacenados)

Procedures son similares a funciones, pero NO retornan valores y soportan transacciones.

Diferencias: Function vs Procedure

Característica	Function	Procedure
Retorno	RETURNS tipo	No retorna
Transacciones	No puede COMMIT/ROLLBACK	Sí puede
Llamada	<code>SELECT funcion()</code>	<code>CALL procedimiento()</code>
Uso en SELECT	Sí	No

Ejemplos de Procedures

1. Procedure Básico

```
sql
```

```
CREATE OR REPLACE PROCEDURE actualizar_precios(  
    incremento_porcentaje DECIMAL  
)  
LANGUAGE plpgsql  
AS $  
BEGIN  
    UPDATE productos  
    SET precio = precio * (1 + incremento_porcentaje / 100);  
  
    RAISE NOTICE 'Precios actualizados con incremento del %', incremento_porcentaje;  
END;  
$;  
  
-- Llamar  
CALL actualizar_precios(10); -- Incrementa todos los precios 10%
```

2. Procedure con Transacciones

sql

```

CREATE OR REPLACE PROCEDURE procesar_ventas_diarias()
LANGUAGE plpgsql
AS $
DECLARE
    venta RECORD;
    errores INTEGER := 0;
BEGIN
    FOR venta IN SELECT * FROM ventas_pendientes LOOP
        BEGIN
            -- Procesar venta
            INSERT INTO ventas_procesadas SELECT * FROM ventas_pendientes WHERE id = venta.id;
            DELETE FROM ventas_pendientes WHERE id = venta.id;

            -- Commit individual por venta
            COMMIT;

        EXCEPTION WHEN OTHERS THEN
            -- Rollback solo esta venta, continuar con las demás
            ROLLBACK;
            errores := errores + 1;
            RAISE NOTICE 'Error procesando venta %: %', venta.id, SQLERRM;
        END;
    END LOOP;

    RAISE NOTICE 'Procesamiento completo. Errores: %', errores;
END;
$;

-- Llamar
CALL procesar_ventas_diarias();

```

3. Procedure con Parámetros OUT

```
sql
```

```

CREATE OR REPLACE PROCEDURE calcular_comisiones(
    mes INTEGER,
    anio INTEGER,
    OUT total_vendedores INTEGER,
    OUT total_comisiones DECIMAL
)
LANGUAGE plpgsql
AS $
BEGIN
    SELECT
        COUNT(DISTINCT vendedor_id),
        SUM(total * 0.05) -- 5% de comisión
    INTO total_vendedores, total_comisiones
    FROM ventas
    WHERE EXTRACT(MONTH FROM fecha) = mes
    AND EXTRACT(YEAR FROM fecha) = anio;

    -- Insertar en tabla de comisiones
    INSERT INTO comisiones_mensuales (mes, anio, total_vendedores, monto_total)
    VALUES (mes, anio, total_vendedores, total_comisiones);

    COMMIT;
END;
$;

-- Llamar
CALL calcular_comisiones(12, 2025, NULL, NULL);

-- O capturar los valores OUT
DO $
DECLARE
    v_vendedores INTEGER;
    v_comisiones DECIMAL;
BEGIN
    CALL calcular_comisiones(12, 2025, v_vendedores, v_comisiones);
    RAISE NOTICE 'Vendedores: %, Comisiones: %', v_vendedores, v_comisiones;
END $;

```

Índices y Optimización

Los índices aceleran las consultas, pero ralentizan escrituras.

Tipos de Índices

1. B-Tree (Por defecto)

Mejor para: $=$, $<$, $>$, $<=$, $>=$, BETWEEN, IN, IS NULL

```
sql

-- Índice simple
CREATE INDEX idx_clientes_email ON clientes(email);

-- Índice compuesto (múltiples columnas)
CREATE INDEX idx_ventas_cliente_fecha ON ventas(cliente_id, fecha);

-- Índice único
CREATE UNIQUE INDEX idx_productos_codigo ON productos(codigo);

-- Índice parcial (solo algunas filas)
CREATE INDEX idx_clientes_activos ON clientes(email)
WHERE activo = true;

-- Índice con INCLUDE (PostgreSQL 11+)
CREATE INDEX idx_productos_categoria_include
ON productos(categoria_id)
INCLUDE (nombre, precio);
```

2. Hash

Mejor para: Solo $=$ (equality)

```
sql

CREATE INDEX idx_hash_productos_codigo
ON productos USING HASH (codigo);
```

3. GIN (Generalized Inverted Index)

Mejor para: Arrays, JSONB, Full-Text Search

```
sql
```

-- Para arrays

```
CREATE INDEX idx_productos_tags ON productos USING GIN (tags);
```

```
SELECT * FROM productos WHERE tags @> ARRAY['electronics', 'laptop'];
```

-- Para JSONB

```
CREATE INDEX idx_clientes_metadata ON clientes USING GIN (metadata_jsonb);
```

```
SELECT * FROM clientes WHERE metadata_jsonb @> '{"pais": "Ecuador"}';
```

-- Para Full-Text Search

```
CREATE INDEX idx_productos_tsvector  
ON productos USING GIN (to_tsvector('spanish', nombre || ' ' || descripcion));
```

```
SELECT * FROM productos  
WHERE to_tsvector('spanish', nombre || ' ' || descripcion)  
@@ to_tsquery('spanish', 'laptop');
```

4. GiST (Generalized Search Tree)

Mejor para: Geometría, rangos, Full-Text Search

sql

-- Para rangos

```
CREATE INDEX idx_eventos_fecha_rango  
ON eventos USING GIST (fecha_rango);
```

```
SELECT * FROM eventos  
WHERE fecha_rango && daterange('2025-01-01', '2025-12-31');
```

-- Para PostGIS (geometría)

```
CREATE INDEX idx_ubicaciones_geom  
ON ubicaciones USING GIST (geom);
```

Análisis de Queries

sql

-- Ver plan de ejecución

```
EXPLAIN SELECT * FROM clientes WHERE email = 'juan@email.com';
```

-- Ver plan CON estadísticas reales

```
EXPLAIN ANALYZE
```

```
SELECT * FROM clientes WHERE email = 'juan@email.com';
```

-- Ver índices de una tabla

```
SELECT
```

```
    indexname,
```

```
    indexdef
```

```
FROM pg_indexes
```

```
WHERE tablename = 'clientes';
```

-- Ver uso de índices

```
SELECT
```

```
    schemaname,
```

```
    tablename,
```

```
    indexname,
```

```
    idx_scan,
```

```
    idx_tup_read,
```

```
    idx_tup_fetch
```

```
FROM pg_stat_user_indexes
```

```
WHERE tablename = 'clientes';
```

-- Ver índices no utilizados

```
SELECT
```

```
    schemaname,
```

```
    tablename,
```

```
    indexname
```

```
FROM pg_stat_user_indexes
```

```
WHERE idx_scan = 0
```

```
AND indexname NOT LIKE '%_pkey';
```

Mantenimiento de Índices

sql


```
-- Reconstruir un índice (elimina bloat)
REINDEX INDEX idx_clientes_email;

-- Reconstruir todos los índices de una tabla
REINDEX TABLE clientes;

-- Reconstruir TODA la base de datos
REINDEX DATABASE quito_db;

-- VACUUM y ANALYZE (mantenimiento rutinario)
VACUUM ANALYZE clientes;

-- VACUUM FULL (reclama espacio, bloquea tabla)
VACUUM FULL clientes;

-- Autovacuum (configurar en postgresql.conf)
autovacuum = on
autovacuum_naptime = 1min
autovacuum_vacuum_scale_factor = 0.2
```

Particionamiento

Divide tablas grandes en "particiones" más pequeñas para mejorar performance.

Tipos de Particionamiento

1. **Range:** Por rangos (fechas, números)
2. **List:** Por lista de valores
3. **Hash:** Por hash de una columna

Particionamiento por Rango (más común)

```
sql
```

-- Tabla principal (particionada)

```
CREATE TABLE ventas (  
  id SERIAL,  
  cliente_id INTEGER,  
  total DECIMAL(10,2),  
  fecha DATE NOT NULL  
) PARTITION BY RANGE (fecha);
```

-- Crear particiones

```
CREATE TABLE ventas_2024_q1 PARTITION OF ventas  
  FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');
```

```
CREATE TABLE ventas_2024_q2 PARTITION OF ventas  
  FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
```

```
CREATE TABLE ventas_2024_q3 PARTITION OF ventas  
  FOR VALUES FROM ('2024-07-01') TO ('2024-10-01');
```

```
CREATE TABLE ventas_2024_q4 PARTITION OF ventas  
  FOR VALUES FROM ('2024-10-01') TO ('2025-01-01');
```

```
CREATE TABLE ventas_2025_q1 PARTITION OF ventas  
  FOR VALUES FROM ('2025-01-01') TO ('2025-04-01');
```

-- Insertar datos (PostgreSQL los pone en la partición correcta automáticamente)

```
INSERT INTO ventas (cliente_id, total, fecha)  
VALUES (1, 150.00, '2024-02-15'); -- Va a ventas_2024_q1
```

```
INSERT INTO ventas (cliente_id, total, fecha)  
VALUES (2, 230.00, '2024-06-20'); -- Va a ventas_2024_q2
```

-- Consultar (PostgreSQL solo busca en las particiones necesarias)

```
SELECT * FROM ventas  
WHERE fecha BETWEEN '2024-01-01' AND '2024-03-31';
```

-- Solo escanea ventas_2024_q1 (partition pruning)

-- Ver particiones

```
SELECT  
  schemaname,  
  tablename,  
  pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS size  
FROM pg_tables  
WHERE tablename LIKE 'ventas%'  
ORDER BY tablename;
```

Particionamiento por Lista

```
sql

-- Particionar por país
CREATE TABLE clientes (
  id SERIAL,
  nombre VARCHAR(100),
  pais VARCHAR(50) NOT NULL
) PARTITION BY LIST (pais);

CREATE TABLE clientes_ecuador PARTITION OF clientes
  FOR VALUES IN ('Ecuador', 'EC');

CREATE TABLE clientes_colombia PARTITION OF clientes
  FOR VALUES IN ('Colombia', 'CO');

CREATE TABLE clientes_peru PARTITION OF clientes
  FOR VALUES IN ('Peru', 'PE');

CREATE TABLE clientes_otros PARTITION OF clientes
  DEFAULT; -- Para valores que no están en otras particiones
```

Particionamiento por Hash

```
sql

-- Particionar por hash del ID (distribuye uniformemente)
CREATE TABLE logs (
  id BIGSERIAL,
  mensaje TEXT,
  fecha TIMESTAMP
) PARTITION BY HASH (id);

-- Crear 4 particiones
CREATE TABLE logs_p0 PARTITION OF logs
  FOR VALUES WITH (MODULUS 4, REMAINDER 0);

CREATE TABLE logs_p1 PARTITION OF logs
  FOR VALUES WITH (MODULUS 4, REMAINDER 1);

CREATE TABLE logs_p2 PARTITION OF logs
  FOR VALUES WITH (MODULUS 4, REMAINDER 2);

CREATE TABLE logs_p3 PARTITION OF logs
  FOR VALUES WITH (MODULUS 4, REMAINDER 3);
```

Sub-Particionamiento

sql

-- Particionar por año, luego por trimestre

```
CREATE TABLE ventas (  
  id SERIAL,  
  total DECIMAL,  
  fecha DATE  
) PARTITION BY RANGE (EXTRACT(YEAR FROM fecha));  
  
CREATE TABLE ventas_2024 PARTITION OF ventas  
  FOR VALUES FROM (2024) TO (2025)  
  PARTITION BY RANGE (EXTRACT(QUARTER FROM fecha));  
  
CREATE TABLE ventas_2024_q1 PARTITION OF ventas_2024  
  FOR VALUES FROM (1) TO (2);  
  
CREATE TABLE ventas_2024_q2 PARTITION OF ventas_2024  
  FOR VALUES FROM (2) TO (3);
```

Mantenimiento de Particiones

sql

-- Agregar nueva partición

```
CREATE TABLE ventas_2025_q2 PARTITION OF ventas
FOR VALUES FROM ('2025-04-01') TO ('2025-07-01');
```

-- Eliminar partición antigua

```
DROP TABLE ventas_2023_q1;
```

-- Detach partición (desconectar sin borrar)

```
ALTER TABLE ventas DETACH PARTITION ventas_2023_q2;
```

-- Attach partición existente

```
ALTER TABLE ventas ATTACH PARTITION ventas_2023_q2
FOR VALUES FROM ('2023-04-01') TO ('2023-07-01');
```

-- Script para crear particiones automáticamente

```
CREATE OR REPLACE FUNCTION crear_particion_ventas(anio INTEGER, trimestre INTEGER)
RETURNS VOID AS $
```

```
DECLARE
```

```
    fecha_inicio DATE;
```

```
    fecha_fin DATE;
```

```
    nombre_particion TEXT;
```

```
BEGIN
```

```
    fecha_inicio := make_date(anio, (trimestre - 1) * 3 + 1, 1);
```

```
    fecha_fin := make_date(anio, trimestre * 3 + 1, 1);
```

```
    nombre_particion := 'ventas_' || anio || '_q' || trimestre;
```

```
    EXECUTE format('CREATE TABLE IF NOT EXISTS %I PARTITION OF ventas FOR VALUES FROM (%L) TO (%L)',
        nombre_particion, fecha_inicio, fecha_fin);
```

```
    RAISE NOTICE 'Partición % creada', nombre_particion;
```

```
END;
```

```
$ LANGUAGE plpgsql;
```

-- Usar

```
SELECT crear_particion_ventas(2026, 1);
```

```
SELECT crear_particion_ventas(2026, 2);
```

Backup y Recuperación

pg_dump (Backup Lógico)

```
bash
```

Backup de una base de datos completa

```
pg_dump -h localhost -U postgres -d quito_db -F c -f backup_quito.dump
```

Explicación de flags:

-h: host

-U: usuario

-d: base de datos

-F c: formato custom (comprimido)

-f: archivo de salida

Backup en formato SQL plano

```
pg_dump -h localhost -U postgres -d quito_db -f backup_quito.sql
```

Backup solo estructura (sin datos)

```
pg_dump -h localhost -U postgres -d quito_db -s -f schema_only.sql
```

Backup solo datos (sin estructura)

```
pg_dump -h localhost -U postgres -d quito_db -a -f data_only.sql
```

Backup de una sola tabla

```
pg_dump -h localhost -U postgres -d quito_db -t clientes -f backup_clientes.sql
```

Backup de múltiples tablas

```
pg_dump -h localhost -U postgres -d quito_db -t clientes -t productos -f backup_tablas.sql
```

Backup excluyendo tablas

```
pg_dump -h localhost -U postgres -d quito_db -T logs -T temp_data -f backup_sin_logs.sql
```

Backup con compresión gzip

```
pg_dump -h localhost -U postgres -d quito_db | gzip > backup_quito.sql.gz
```

pg_dumpall (Todo el cluster)

bash

Backup de TODAS las bases de datos + roles

```
pg_dumpall -h localhost -U postgres -f backup_completo.sql
```

Solo roles y tablespaces

```
pg_dumpall -h localhost -U postgres -g -f backup_roles.sql
```

Restaurar Backups

bash

```
# Restaurar desde formato custom
pg_restore -h localhost -U postgres -d quito_db backup_quito.dump

# Restaurar desde SQL
psql -h localhost -U postgres -d quito_db -f backup_quito.sql

# Restaurar creando la BD primero
createdb -h localhost -U postgres quito_db
pg_restore -h localhost -U postgres -d quito_db backup_quito.dump

# Restaurar solo una tabla
pg_restore -h localhost -U postgres -d quito_db -t clientes backup_quito.dump

# Restaurar sin constraints (para datos problemáticos)
pg_restore -h localhost -U postgres -d quito_db --disable-triggers backup_quito.dump
```

PITR (Point-In-Time Recovery)

Permite recuperar la BD a un momento exacto en el tiempo.

```
bash

# 1. Configurar en postgresql.conf
wal_level = replica
archive_mode = on
archive_command = 'cp %p /ruta/a/archivo/wal/%f'
max_wal_senders = 3

# 2. Hacer base backup
pg_basebackup -h localhost -U replicador -D /backup/base -P -v

# 3. Para recuperar a un punto específico
# Copiar base backup a directorio de datos
cp -r /backup/base/* /var/lib/postgresql/14/main/

# Crear recovery.signal
touch /var/lib/postgresql/14/main/recovery.signal

# Configurar recovery en postgresql.auto.conf
restore_command = 'cp /ruta/a/archivo/wal/%f %p'
recovery_target_time = '2025-12-12 14:30:00'

# Iniciar PostgreSQL
systemctl start postgresql
```

Seguridad y Permisos

Crear Usuarios y Roles

```
sql

-- Crear usuario
CREATE USER usuario1 WITH PASSWORD 'password123';

-- Crear rol (grupo de permisos)
CREATE ROLE empleados;

-- Crear usuario con opciones
CREATE USER admin1 WITH
    PASSWORD 'securepass'
    CREATEDB
    CREATEROLE
    LOGIN;

-- Crear rol con permisos específicos
CREATE ROLE readonly
    NOLOGIN
    NOSUPERUSER
    NOCREATEDB;

-- Agregar usuario a rol
GRANT empleados TO usuario1;

-- Ver roles y usuarios
\du

-- O con query
SELECT rolname, rolsuper, rolcreatedb, rolcreaterole, rolcanlogin
FROM pg_roles
ORDER BY rolname;
```

Permisos Granulares

```
sql
```


-- Dar todos los permisos en una BD

```
GRANT ALL PRIVILEGES ON DATABASE quito_db TO usuario1;
```

-- Dar permisos en un schema

```
GRANT USAGE ON SCHEMA public TO empleados;
```

-- Dar permisos en tablas

```
GRANT SELECT ON clientes TO empleados;
```

```
GRANT SELECT, INSERT, UPDATE ON productos TO empleados;
```

```
GRANT ALL PRIVILEGES ON ventas TO admin1;
```

-- Dar permisos en TODAS las tablas de un schema

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly;
```

-- Dar permisos en tablas FUTURAS

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public
```

```
GRANT SELECT ON TABLES TO readonly;
```

-- Permisos en secuencias (para SERIAL/IDENTITY)

```
GRANT USAGE ON SEQUENCE clientes_id_seq TO usuario1;
```

```
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO empleados;
```

-- Permisos en funciones

```
GRANT EXECUTE ON FUNCTION calcular_iva(DECIMAL) TO empleados;
```

-- Revocar permisos

```
REVOKE INSERT, UPDATE, DELETE ON clientes FROM usuario1;
```

-- Ver permisos de una tabla

```
\dp clientes
```

-- O con query

```
SELECT
```

```
grantee,
```

```
privilege_type
```

```
FROM information_schema.role_table_grants
```

```
WHERE table_name = 'clientes';
```

Row Level Security (RLS)

Controla qué FILAS puede ver cada usuario.

```
sql
```

-- Habilitar RLS en la tabla

```
ALTER TABLE clientes ENABLE ROW LEVEL SECURITY;
```

-- Política: usuarios solo ven sus propios clientes

```
CREATE POLICY clientes_policy ON clientes
  FOR SELECT
  TO empleados
  USING (vendedor_id = current_user_id());
```

-- Política: gerentes ven todo

```
CREATE POLICY gerentes_policy ON clientes
  FOR ALL
  TO gerentes
  USING (true);
```

-- Política: usuarios pueden insertar solo con su vendedor_id

```
CREATE POLICY clientes_insert_policy ON clientes
  FOR INSERT
  TO empleados
  WITH CHECK (vendedor_id = current_user_id());
```

-- Deshabilitar temporalmente RLS

```
ALTER TABLE clientes DISABLE ROW LEVEL SECURITY;
```

-- Ver políticas

```
\d+ clientes
```

-- Eliminar política

```
DROP POLICY clientes_policy ON clientes;
```

Encriptación

sql

```
-- Instalar extensión pgcrypto
CREATE EXTENSION pgcrypto;

-- Hashear contraseñas
INSERT INTO usuarios (username, password_hash)
VALUES ('juan', crypt('mipassword', gen_salt('bf')));

-- Verificar contraseña
SELECT * FROM usuarios
WHERE username = 'juan'
AND password_hash = crypt('mipassword', password_hash);

-- Encriptar datos sensibles
CREATE TABLE tarjetas (
  id SERIAL PRIMARY KEY,
  numero_encriptado BYTEA,
  cliente_id INTEGER
);

-- Insertar encriptado
INSERT INTO tarjetas (numero_encriptado, cliente_id)
VALUES (
  pgp_sym_encrypt('4532-1234-5678-9010', 'clave_secreta'),
  123
);

-- Consultar desencriptado
SELECT
  id,
  pgp_sym_decrypt(numero_encriptado, 'clave_secreta') AS numero,
  cliente_id
FROM tarjetas;
```

Monitoreo y Mantenimiento

Queries de Monitoreo

```
sql
```

-- Ver conexiones activas

SELECT

pid,
username,
application_name,
client_addr,
state,
query_start,
query

FROM pg_stat_activity

WHERE state = 'active'

ORDER BY query_start;

-- Ver queries lentas

SELECT

pid,
now() - query_start AS duration,
query

FROM pg_stat_activity

WHERE state = 'active'

AND now() - query_start > interval '5 minutes';

-- Matar query lenta

SELECT pg_cancel_backend(12345); -- PID del query

-- Terminar conexión

SELECT pg_terminate_backend(12345);

-- Ver tamaño de bases de datos

SELECT

datname,
pg_size_pretty(pg_database_size(datname)) AS size

FROM pg_database

ORDER BY pg_database_size(datname) DESC;

-- Ver tamaño de tablas

SELECT

schemaname,
tablename,
pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS size,
pg_size_pretty(pg_relation_size(schemaname||'.'||tablename)) AS table_size,
pg_size_pretty(pg_indexes_size(schemaname||'.'||tablename)) AS indexes_size

FROM pg_tables

WHERE schemaname NOT IN ('pg_catalog', 'information_schema')

ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC

LIMIT 20;

-- Ver cache hit ratio (debería ser > 90%)

```
SELECT
    sum(heap_blks_read) AS heap_read,
    sum(heap_blks_hit) AS heap_hit,
    round(sum(heap_blks_hit) / (sum(heap_blks_hit) + sum(heap_blks_read)) * 100, 2) AS cache_hit_ratio
FROM pg_statio_user_tables;
```

-- Ver estadísticas de tablas

```
SELECT
    schemaname,
    tablename,
    n_tup_ins AS inserts,
    n_tup_upd AS updates,
    n_tup_del AS deletes,
    n_live_tup AS live_tuples,
    n_dead_tup AS dead_tuples,
    last_vacuum,
    last_autovacuum,
    last_analyze,
    last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

-- Ver tablas con mucho bloat

```
SELECT
    schemaname,
    tablename,
    n_dead_tup,
    n_live_tup,
    round(n_dead_tup * 100.0 / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_ratio
FROM pg_stat_user_tables
WHERE n_dead_tup > 1000
ORDER BY dead_ratio DESC;
```

-- Ver locks activos

```
SELECT
    pid,
    username,
    pg_blocking_pids(pid) AS blocked_by,
    query AS blocked_query
FROM pg_stat_activity
WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

-- Ver estadísticas de índices

```
SELECT
    schemaname,
```

```
tablename,  
indexname,  
idx_scan AS scans,  
idx_tup_read AS tuples_read,  
idx_tup_fetch AS tuples_fetched  
FROM pg_stat_user_indexes  
ORDER BY idx_scan DESC;  
  
-- Ver checkpoint stats  
SELECT * FROM pg_stat_bgwriter;  
  
-- Ver replicación  
SELECT * FROM pg_stat_replication;
```

Configuraciones Importantes (postgresql.conf)

```
ini
```

MEMORIA

`shared_buffers = 256MB` *# 25% de RAM (máximo 8GB)*
`effective_cache_size = 1GB` *# 50-75% de RAM total*
`work_mem = 4MB` *# Por operación de sort/hash*
`maintenance_work_mem = 64MB` *# Para VACUUM, CREATE INDEX*

CHECKPOINT

`checkpoint_timeout = 15min`
`checkpoint_completion_target = 0.9`
`max_wal_size = 2GB`
`min_wal_size = 1GB`

LOGS

`log_destination = 'stderr'`
`logging_collector = on`
`log_directory = 'log'`
`log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'`
`log_rotation_age = 1d`
`log_rotation_size = 100MB`
`log_min_duration_statement = 1000` *# Log queries > 1 segundo*
`log_line_prefix = '%t [%p]: user=%u,db=%d,app=%a,client=%h '`
`log_checkpoints = on`
`log_connections = on`
`log_disconnections = on`
`log_lock_waits = on`

AUTOVACUUM

`autovacuum = on`
`autovacuum_max_workers = 3`
`autovacuum_naptime = 1min`
`autovacuum_vacuum_scale_factor = 0.2`
`autovacuum_analyze_scale_factor = 0.1`

CONEXIONES

`max_connections = 100`
`superuser_reserved_connections = 3`

REPLICACIÓN

`wal_level = replica`
`max_wal_senders = 5`
`max_replication_slots = 5`
`wal_keep_size = 1GB`

QUERY PLANNING

```
random_page_cost = 1.1          # Para SSD (default 4.0 es para HDD)
effective_io_concurrency = 200   # Para SSD
```

Scripts de Mantenimiento

```
sql

-- Script diario de mantenimiento
CREATE OR REPLACE PROCEDURE mantenimiento_diario()
LANGUAGE plpgsql
AS $
BEGIN
    -- VACUUM y ANALYZE de todas las tablas
    VACUUM ANALYZE;

    -- Reindexar tablas con mucho bloat
    REINDEX TABLE CONCURRENTLY clientes;
    REINDEX TABLE CONCURRENTLY productos;

    -- Limpiar logs antiguos (si tienes tabla de logs)
    DELETE FROM logs WHERE fecha < NOW() - INTERVAL '30 days';

    COMMIT;

    RAISE NOTICE 'Mantenimiento diario completado';
END;
$;

-- Programar con cron o pg_cron
-- Instalar extensión pg_cron
CREATE EXTENSION pg_cron;

-- Agendar mantenimiento diario a las 2 AM
SELECT cron.schedule('mantenimiento-diario', '0 2 * * *', 'CALL mantenimiento_diario()');

-- Ver jobs programados
SELECT * FROM cron.job;

-- Eliminar job
SELECT cron.unschedule(1); -- ID del job
```

Análisis de Performance

```
sql
```



```
-- Habilitar pg_stat_statements (en postgresql.conf)
```

```
-- shared_preload_libraries = 'pg_stat_statements'
```

```
-- Instalar extensión
```

```
CREATE EXTENSION pg_stat_statements;
```

```
-- Ver queries más lentas
```

```
SELECT
```

```
    query,  
    calls,  
    total_exec_time,  
    mean_exec_time,  
    max_exec_time,  
    rows
```

```
FROM pg_stat_statements
```

```
ORDER BY mean_exec_time DESC
```

```
LIMIT 20;
```

```
-- Resetear estadísticas
```

```
SELECT pg_stat_statements_reset();
```

```
-- Ver queries más frecuentes
```

```
SELECT
```

```
    query,  
    calls,  
    total_exec_time,  
    mean_exec_time
```

```
FROM pg_stat_statements
```

```
ORDER BY calls DESC
```

```
LIMIT 20;
```

```
-- Ver queries que más consumen CPU
```

```
SELECT
```

```
    query,  
    calls,  
    total_exec_time,  
    100.0 * total_exec_time / sum(total_exec_time) OVER () AS percentage
```

```
FROM pg_stat_statements
```

```
ORDER BY total_exec_time DESC
```

```
LIMIT 20;
```

Best Practices y Tips de DBA

1. Naming Conventions

sql

-- TABLAS: plural, snake_case

CREATE TABLE clientes (...);

CREATE TABLE ordenes_compra (...);

-- COLUMNAS: singular, snake_case

CREATE TABLE clientes (

id SERIAL PRIMARY KEY,

nombre_completo VARCHAR(200),

fecha_nacimiento DATE,

email VARCHAR(100)

);

-- ÍNDICES: idx_tabla_columna(s)

CREATE INDEX idx_clientes_email ON clientes(email);

CREATE INDEX idx_ventas_cliente_fecha ON ventas(cliente_id, fecha);

-- FOREIGN KEYS: fk_tabla_columna

ALTER TABLE ventas

ADD CONSTRAINT fk_ventas_cliente

FOREIGN KEY (cliente_id) REFERENCES clientes(id);

-- TRIGGERS: trg_tabla_momento_accion

CREATE TRIGGER trg_clientes_before_insert ...;

-- FUNCIONES: verbo_sustantivo

CREATE FUNCTION calcular_total(...);

CREATE FUNCTION obtener_clientes_activos(...);

2. Siempre Usar Transacciones

sql

-- ❌ MAL: Sin transacción

UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;

UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;

-- Si falla la segunda, quedas inconsistente

-- ✅ BIEN: Con transacción

BEGIN;

UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;

UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;

COMMIT;

-- Si falla cualquiera, todo se revierte

3. Usar EXPLAIN ANALYZE

```
sql

-- Antes de crear un índice, verificar que lo usará
EXPLAIN ANALYZE
SELECT * FROM ventas WHERE cliente_id = 123;

-- Si ves "Seq Scan" en tablas grandes, necesitas índice
-- Si ves "Index Scan", está usando el índice correctamente
```

4. No Usar SELECT *

```
sql

-- ❌ MAL: Trae datos innecesarios
SELECT * FROM clientes;

-- ✅ BIEN: Solo las columnas necesarias
SELECT id, nombre, email FROM clientes;
```

5. Indexes en Foreign Keys

```
sql

-- ✅ SIEMPRE indexar foreign keys
CREATE INDEX idx_ventas_cliente ON ventas(cliente_id);
CREATE INDEX idx_ventas_producto ON ventas(producto_id);
```

6. Connection Pooling

```
bash

# Usar PgBouncer o similar en producción
# postgresql.conf
max_connections = 100 # No subir demasiado

# Usar connection pooling en la aplicación
# Ejemplo con PgBouncer
[databases]
quito_db = host=localhost port=5432 dbname=quito_db

[pgbouncer]
pool_mode = transaction
max_client_conn = 1000
default_pool_size = 25
```

7. Monitorear Constantemente

sql

-- Script de health check

CREATE OR REPLACE FUNCTION health_check()

RETURNS TABLE(
 check_name TEXT,
 status TEXT,
 value TEXT
) AS \$

BEGIN

-- Cache hit ratio

RETURN QUERY

SELECT

'Cache Hit Ratio'::TEXT,

CASE WHEN ratio > 90 THEN 'OK' ELSE 'WARNING' END,

ratio::TEXT || '%'

FROM (

SELECT round(sum(heap_blks_hit) * 100.0 / NULLIF(sum(heap_blks_hit) + sum(heap_blks_read), 0), 2) AS ratio

FROM pg_statio_user_tables

) t;

-- Conexiones

RETURN QUERY

SELECT

'Active Connections'::TEXT,

CASE WHEN count(*) > 80 THEN 'WARNING' ELSE 'OK' END,

count(*)::TEXT

FROM pg_stat_activity

WHERE state = 'active';

-- Bloat

RETURN QUERY

SELECT

'Tables with High Bloat'::TEXT,

CASE WHEN count(*) > 5 THEN 'WARNING' ELSE 'OK' END,

count(*)::TEXT

FROM pg_stat_user_tables

WHERE n_dead_tup > n_live_tup * 0.2;

-- Replication lag (si aplica)

RETURN QUERY

SELECT

'Replication Lag'::TEXT,

CASE WHEN EXTRACT(EPOCH FROM lag) > 60 THEN 'WARNING' ELSE 'OK' END,

lag::TEXT

FROM (

SELECT COALESCE(now() - pg_last_xact_replay_timestamp(), interval '0') AS lag

) t;

```
END;  
$ LANGUAGE plpgsql;  
  
-- Ejecutar  
SELECT * FROM health_check();
```

8. Backups Automatizados

```
bash  
  
#!/bin/bash  
# Script: backup_diario.sh  
  
BACKUP_DIR="/backups/postgresql"  
DATE=$(date +%Y%m%d_%H%M%S)  
DB_NAME="quito_db"  
  
# Crear directorio si no existe  
mkdir -p $BACKUP_DIR  
  
# Backup  
pg_dump -h localhost -U postgres -d $DB_NAME -F c -f $BACKUP_DIR/backup_${DB_NAME}_${DATE}.dump  
  
# Limpiar backups antiguos (más de 7 días)  
find $BACKUP_DIR -name "*.dump" -mtime +7 -delete  
  
# Verificar que el backup se creó correctamente  
if [ $? -eq 0 ]; then  
    echo "Backup exitoso: backup_${DB_NAME}_${DATE}.dump"  
else  
    echo "ERROR en backup" | mail -s "Error Backup PostgreSQL" admin@empresa.com  
fi  
  
# Agendar en crontab:  
# 0 2 * * * /path/to/backup_diario.sh
```

Casos de Uso Reales

1. Sistema de E-Commerce

```
sql
```

-- Estructura completa

```
CREATE TABLE categorias (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  descripcion TEXT,  
  activo BOOLEAN DEFAULT true  
);
```

```
CREATE TABLE productos (  
  id SERIAL PRIMARY KEY,  
  codigo VARCHAR(50) UNIQUE NOT NULL,  
  nombre VARCHAR(200) NOT NULL,  
  descripcion TEXT,  
  precio DECIMAL(10,2) NOT NULL CHECK (precio > 0),  
  stock INTEGER DEFAULT 0 CHECK (stock >= 0),  
  categoria_id INTEGER REFERENCES categorias(id),  
  activo BOOLEAN DEFAULT true,  
  fecha_creacion TIMESTAMP DEFAULT NOW(),  
  fecha_modificacion TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE clientes (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  telefono VARCHAR(20),  
  direccion TEXT,  
  total_compras INTEGER DEFAULT 0,  
  monto_total DECIMAL(10,2) DEFAULT 0,  
  fecha_registro TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE ventas (  
  id SERIAL PRIMARY KEY,  
  cliente_id INTEGER REFERENCES clientes(id),  
  subtotal DECIMAL(10,2) NOT NULL,  
  descuento DECIMAL(10,2) DEFAULT 0,  
  iva DECIMAL(10,2) NOT NULL,  
  total DECIMAL(10,2) NOT NULL,  
  estado VARCHAR(20) DEFAULT 'PENDIENTE',  
  fecha TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE ventas_detalle (  
  id SERIAL PRIMARY KEY,  
  venta_id INTEGER REFERENCES ventas(id) ON DELETE CASCADE,
```

```
producto_id INTEGER REFERENCES productos(id),
cantidad INTEGER NOT NULL CHECK (cantidad > 0),
precio_unitario DECIMAL(10,2) NOT NULL,
subtotal DECIMAL(10,2) NOT NULL
);
```

-- Índices

```
CREATE INDEX idx_productos_categoria ON productos(categoria_id);
CREATE INDEX idx_productos_activo ON productos(activo) WHERE activo = true;
CREATE INDEX idx_ventas_cliente ON ventas(cliente_id);
CREATE INDEX idx_ventas_fecha ON ventas(fecha);
CREATE INDEX idx_ventas_detalle_venta ON ventas_detalle(venta_id);
CREATE INDEX idx_ventas_detalle_producto ON ventas_detalle(producto_id);
```

-- Función: Procesar venta completa

```
CREATE OR REPLACE FUNCTION procesar_venta(
    p_cliente_id INTEGER,
    p_items JSONB -- [{"producto_id": 1, "cantidad": 2}, ...]
) RETURNS INTEGER AS $
DECLARE
    v_venta_id INTEGER;
    v_subtotal DECIMAL := 0;
    v_iva DECIMAL;
    v_total DECIMAL;
    v_item JSONB;
    v_precio DECIMAL;
    v_stock INTEGER;
BEGIN
    -- Validar items
    IF jsonb_array_length(p_items) = 0 THEN
        RAISE EXCEPTION 'La venta debe tener al menos un producto';
    END IF;

    -- Calcular subtotal y validar stock
    FOR v_item IN SELECT * FROM jsonb_array_elements(p_items)
    LOOP
        SELECT precio, stock INTO v_precio, v_stock
        FROM productos
        WHERE id = (v_item->>'producto_id')::INTEGER
        AND activo = true;

        IF NOT FOUND THEN
            RAISE EXCEPTION 'Producto % no existe o está inactivo', v_item->>'producto_id';
        END IF;

        IF v_stock < (v_item->>'cantidad')::INTEGER THEN
            RAISE EXCEPTION 'Stock insuficiente para producto %. Disponible: %',
```



```

        v_item->>'producto_id', v_stock;
    END IF;

    v_subtotal := v_subtotal + (v_precio * (v_item->>'cantidad')::INTEGER);
END LOOP;

-- Calcular IVA y total
v_iva := v_subtotal * 0.15;
v_total := v_subtotal + v_iva;

-- Crear venta
INSERT INTO ventas (cliente_id, subtotal, descuento, iva, total, estado)
VALUES (p_cliente_id, v_subtotal, 0, v_iva, v_total, 'COMPLETADA')
RETURNING id INTO v_venta_id;

-- Insertar detalles y actualizar stock
FOR v_item IN SELECT * FROM jsonb_array_elements(p_items)
LOOP
    SELECT precio INTO v_precio
    FROM productos
    WHERE id = (v_item->>'producto_id')::INTEGER;

    INSERT INTO ventas_detalle (venta_id, producto_id, cantidad, precio_unitario, subtotal)
    VALUES (
        v_venta_id,
        (v_item->>'producto_id')::INTEGER,
        (v_item->>'cantidad')::INTEGER,
        v_precio,
        v_precio * (v_item->>'cantidad')::INTEGER
    );

    UPDATE productos
    SET stock = stock - (v_item->>'cantidad')::INTEGER
    WHERE id = (v_item->>'producto_id')::INTEGER;
END LOOP;

-- Actualizar estadísticas del cliente
UPDATE clientes
SET
    total_compras = total_compras + 1,
    monto_total = monto_total + v_total
WHERE id = p_cliente_id;

RETURN v_venta_id;
END;
$ LANGUAGE plpgsql;

```

```
-- Uso
SELECT procesar_venta(
  1, -- cliente_id
  '[{"producto_id": 1, "cantidad": 2}, {"producto_id": 3, "cantidad": 1}]'::JSONB
);
```

2. Sistema de Auditoría Universal

sql

-- Tabla de auditoría

```
CREATE TABLE auditoria (  
    id BIGSERIAL PRIMARY KEY,  
    tabla VARCHAR(100),  
    operacion VARCHAR(10),  
    registro_id INTEGER,  
    usuario VARCHAR(100),  
    datos_anteriores JSONB,  
    datos_nuevos JSONB,  
    ip_address INET,  
    fecha TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE INDEX idx_auditoria_tabla ON auditoria(tabla);  
CREATE INDEX idx_auditoria_fecha ON auditoria(fecha);  
CREATE INDEX idx_auditoria_usuario ON auditoria(usuario);
```

-- Función trigger universal

```
CREATE OR REPLACE FUNCTION trg_auditoria_universal()  
RETURNS TRIGGER AS $  
BEGIN  
    INSERT INTO auditoria (  
        tabla,  
        operacion,  
        registro_id,  
        usuario,  
        datos_anteriores,  
        datos_nuevos,  
        ip_address  
    ) VALUES (  
        TG_TABLE_NAME,  
        TG_OP,  
        COALESCE(NEW.id, OLD.id),  
        current_user,  
        CASE WHEN TG_OP != 'INSERT' THEN row_to_json(OLD)::JSONB ELSE NULL END,  
        CASE WHEN TG_OP != 'DELETE' THEN row_to_json(NEW)::JSONB ELSE NULL END,  
        inet_client_addr()  
    );  
  
    IF TG_OP = 'DELETE' THEN  
        RETURN OLD;  
    ELSE  
        RETURN NEW;  
    END IF;  
END;  
$ LANGUAGE plpgsql;
```

-- Aplicar a tablas importantes

```
CREATE TRIGGER auditoria_clientes
AFTER INSERT OR UPDATE OR DELETE ON clientes
FOR EACH ROW EXECUTE FUNCTION trg_auditoria_universal();
```

```
CREATE TRIGGER auditoria_productos
AFTER INSERT OR UPDATE OR DELETE ON productos
FOR EACH ROW EXECUTE FUNCTION trg_auditoria_universal();
```

-- Consultar auditoría

```
SELECT
    tabla,
    operacion,
    usuario,
    datos_anteriores->>'nombre' AS nombre_anterior,
    datos_nuevos->>'nombre' AS nombre_nuevo,
    fecha
FROM auditoria
WHERE tabla = 'clientes'
AND registro_id = 5
ORDER BY fecha DESC;
```

Glosario de Términos DBA

- **WAL (Write-Ahead Log):** Log de transacciones para recuperación
 - **VACUUM:** Proceso que libera espacio de filas eliminadas
 - **ANALYZE:** Actualiza estadísticas para el query planner
 - **Bloat:** Espacio desperdiciado por filas muertas
 - **Checkpoint:** Punto donde se escribe data del buffer a disco
 - **Replication Slot:** Mecanismo para tracking de replicación
 - **LSN (Log Sequence Number):** Posición en el WAL
 - **Hot Standby:** Replica que acepta queries de lectura
 - **PITR:** Point-In-Time Recovery
 - **Connection Pooling:** Reutilización de conexiones
 - **Partition Pruning:** Optimización que ignora particiones irrelevantes
 - **MVCC (Multi-Version Concurrency Control):** Sistema de transacciones de PostgreSQL
-

Recursos Adicionales

Documentación Oficial

- <https://www.postgresql.org/docs/>
- <https://wiki.postgresql.org/>

Herramientas Útiles

- **pgAdmin:** GUI para administración
- **DBeaver:** Cliente universal
- **pg_stat_statements:** Análisis de queries
- **PgBouncer:** Connection pooler
- **Patroni:** High availability
- **pg_cron:** Cron jobs dentro de PostgreSQL
- **TimescaleDB:** Extensión para time-series data

Comandos psql Útiles

```
bash

\l           # Listar bases de datos
\c dbname    # Conectar a BD
\dt          # Listar tablas
\d+ tablename # Describir tabla
\df          # Listar funciones
\du          # Listar usuarios
\dx          # Listar extensiones
\timing on   # Activar timing de queries
\x           # Modo expanded (vertical)
\g           # Ejecutar último comando
\q           # Salir
```

Checklist del DBA PostgreSQL

Diario

- ☐ Revisar logs de errores
- ☐ Verificar conexiones activas
- ☐ Monitorear queries lentas

- ☐ Revisar espacio en disco

Semanal

- ☐ Analizar performance de queries
- ☐ Revisar índices no utilizados
- ☐ Verificar estado de replicación
- ☐ Revisar tamaño de tablas y bloat

Mensual

- ☐ Revisar y ajustar configuración
- ☐ Analizar tendencias de crecimiento
- ☐ Planificar particionamiento si es necesario
- ☐ Actualizar documentación

Trimestral

- ☐ Evaluar upgrades de versión
- ☐ Revisar plan de disaster recovery
- ☐ Optimizar queries más costosas
- ☐ Capacitación del equipo

¡Eso es todo amigo! Con esta guía tienes TODO lo necesario para convertirte en un DBA PostgreSQL profesional. 🚀

Próximos pasos recomendados:

1. Practica cada concepto en un ambiente de desarrollo
2. Crea un proyecto real que uses diariamente
3. Contribuye a proyectos open source que usen PostgreSQL
4. Obtén certificaciones oficiales

¡Mucha suerte en tu camino a DBA! 💪